

Motus — Tout ce qu'on a fait, à quoi ça sert & comment

Récapitulatif complet du projet · M2 MIAGE SITN · Liya & Steven

Cette fiche liste chaque fonctionnalité du projet, avec pour chacune : à quoi ça sert (le pourquoi) et comment on l'a faite (le comment technique). Elle sert aussi de check-list des consignes.

1 L'architecture microservices

Service	Port	Rôle	Base MySQL	Qui
player-service	8081	Joueurs : inscription, connexion	motus_players	Steven
game-service	8082	Parties : algo Motus, essais	motus_games	Steven
dict-service	8083	Dictionnaire : mot aléatoire, validation	motus_dictionary	Liya
score-service	8084	Scores : calcul, classement, stats	motus_scores	Liya
frontend	5500	UI HTML/CSS/JS servi par un http.server	—	Liya

Pourquoi ? Une base par service, communication REST (HTTP + JSON), chaque service déployable indépendamment. C'est la définition des microservices (Martin Fowler) et le cœur du cours.

2 Le jeu (côté joueur)

Choix du niveau (5 à 9 lettres)

À quoi ça sert : le joueur choisit la difficulté ; plus long = plus dur = plus de points.

Comment : le front appelle `GET /words/lengths` (dict). Le repo Spring Data JPA filtre par `WHERE longueur BETWEEN 5 AND 9` pour ne pas exposer les mots de 4 lettres (trop faciles). Les 5 cartes sont affichées sur une ligne avec un code couleur (bleu → rouge).

Grille Motus + clavier AZERTY interactif

À quoi ça sert : vivre le jeu comme le vrai Motus (grille, animations, feedback couleur).

Comment : grille HTML/CSS générée en JS ; chaque case a un état (`hint` , `filled` , `correct` , `present` , `absent`). Animation `flip` à la révélation, `shake` quand le mot est invalide.

Première lettre en indice fantôme

À quoi ça sert : aide sans faire à la place. Le joueur doit quand même la taper.

Comment : le back `game-service` renvoie `"M****"` pendant la partie ; le front affiche le M en vert transparent italique (`.tile.hint`). Il redevient invisible quand on tape.

Le pendu (potence + mascotte KO)

À quoi ça sert : retour visuel des vies restantes, ambiance ludique.

Comment : SVG à 5 stades (socle, poteau, poutre, corde, mascotte). Chaque erreur ajoute une pièce (fonction `updatePendu()`). À la défaite, la mascotte passe en visage KO (yeux X, étoiles, balancement CSS).

Écran de fin : mascotte souriante (victoire) ou KO (défaite)

À quoi ça sert : récompense visuelle claire — plus lisible qu'un emoji.

Comment : `HAPPY_OWL_SVG` (couronne dorée + grands yeux + sourire) vs `KO_OWL_SVG` (yeux en croix, langue, étoiles). Confettis en JS pour la victoire. Bouton Définition du mot qui redirige vers Larousse.

3 Compte utilisateur : inscription, connexion, invité

Inscription

À quoi ça sert : avoir un profil persistant avec pseudo unique, e-mail, scores enregistrés.

Comment : modale d'auth (onglet "Créer un compte") → `POST /players {username, email, password}` . Le mot de passe est haché par **BCrypt** côté player-service avant stockage MySQL.

Connexion

À quoi ça sert : retrouver son compte pour continuer à cumuler les scores.

Comment : nouvel endpoint `POST /players/login {identifiant, password}` . Accepte le pseudo OU l'email. Vérifie le hash avec `passwordEncoder.matches()` . 401 si mauvais mot de passe. Le front stocke la session en `localStorage` .

Mode invité

À quoi ça sert : essayer le jeu sans créer de compte. Compromis : pas d'historique, pas d'apparition dans le classement.

Comment : bouton "Jouer en invité" → `state.isGuest = true` , `playerId=0` envoyé au back. Les tool-buttons `.need-account` (stats/historique) sont cachés. Le classement filtre les parties avec `playerId <= 0` côté score-service. Le chip "Invité →" ouvre directement l'écran de connexion (pas de menu déconnexion).

4 Classement · Historique · Statistiques

Classement global (avec 12 joueurs de démo)

À quoi ça sert : compétition entre joueurs, motive à rejouer.

Comment : `GET /scores/ranking` → trié par `totalScore` desc, puis par nombre de victoires. Le front récupère les vrais pseudos via `resolveNames()` qui appelle `GET /players/{id}` pour chaque ID inconnu (cache local). Podium avec médailles 🥇 🥈 🥉, labels clairs (🎮 X parties · 🏆 Y victoires · 📊 Z%). Ta ligne est mise en évidence en vert.

Historique de tes parties

À quoi ça sert : voir ses parties passées, le mot deviné, le score, revoir la définition.

Comment : `GET /scores/games?playerId=X` → parties triées par date. Pour chaque partie, on récupère le mot en parallèle via `GET /games/{gameId}`. Badge propre gagné/perdu (couronne verte / trait rouge), mot en gros, date + durée + essais, lien Larousse.

Statistiques personnelles

À quoi ça sert : mesurer ses progrès (parties, victoires, taux, moyenne, meilleur score).

Comment : `GET /scores/players/{id}` agrège les parties du joueur en 6 stats. Rendu en grille de tuiles avec valeurs et libellés.

Formule de score

À quoi ça sert : valoriser la vitesse, l'économie d'essais et surtout la longueur du mot.

Comment : $\text{score} = (100 + (\text{max-essais}) \times 20 + \max(0, 50 - \text{durée}/5)) \times (\text{longueur} - 2)$. La longueur MULTIPLIE tout le score. 0 si perdu. Idempotent par gameld (pas de double comptage si le back rejoue le push).

5 Partager le jeu à ses amis

Trois façons de partager

À quoi ça sert : viraliser le jeu, jouer avec ses amis.

Comment : bouton dans le header ouvre une modale avec :

- Copier le lien — utilise `navigator.clipboard.writeText`.
- WhatsApp — lien `https://wa.me/?text=...` avec un message pré-rédigé personnalisé (« Coucou ! (Liya) Je t'invite à jouer à Motus avec moi 🎮 ... »).
- QR code — généré côté client avec la lib open-source `qrcodejs` (CDN Cloudflare), aux couleurs Motus. Scannable au téléphone.

6 Un dictionnaire propre

132 000 mots français, sans prénoms ni "mots bizarres"

À quoi ça sert : le jeu ne tire jamais un prénom (« TOMAS ») ou une conjugaison difficile (« MANGERONS ») comme mot mystère.

Comment : deux couches.

1. Nettoyage BDD au démarrage — `data-cleanup.sql` supprime une liste noire de ~110 prénoms (Tomas, Julien, Marie, Steven, Lucas, Ines, Chloe...).
2. Filtre au tirage — la requête `findRandomJouableByLongueur` exclut toutes les terminaisons de conjugaison (EZ, ERAI, ERONS, ERAIT, ERAIENT, ASSE, ISSE, USSE...) avec une whitelist des noms courants (SUISSE, CAISSE, MOUSSE, ASSEZ, CHEZ...).

Important : les conjugaisons restent en BDD et sont acceptées comme *proposition du joueur* (via `existsByMotIgnoreCase`), mais elles ne peuvent pas être choisies comme réponse.

7 La stack technique (ce que le prof cherchera)

Spring Boot 4.0.7 · Java 26

Les 4 services sont des applications Spring Boot autonomes avec Tomcat embarqué. Le `pom.xml` déclare les starters (`web` , `data-jpa` , `actuator` , driver MySQL).

ORM · JPA · Hibernate

Entités `@Entity` (Mot, GameResult, Player, Game) avec `@Id` `@GeneratedValue @Column` . Repositories `JpaRepository` avec query methods (`findByLongueur` , `existsByMotIgnoreCase`). Hibernate génère le SQL.

REST (Richardson niveau 2)

URI par ressource (`/words/random` , `/scores/ranking`), verbes HTTP (GET/POST), codes standard (200, 201, 400, 401, 404, 409). JSON en camelCase.

Communication inter-services

RestTemplate (`DictServiceClient` , `ScoreServiceClient`). Modèle **push** : game-service pousse le résultat vers score-service en fin de partie. Idempotent par `gameId` .

MySQL 8 (une base par service)

Chaque service a sa propre base créée automatiquement (`createDatabaseIfNotExist=true`). Persistance entre redémarrages, seed data avec `INSERT IGNORE` (idempotent).

Docker + docker-compose

Dockerfile multi-étapes (build Maven puis JRE léger). `docker-compose.yml` orchestre MySQL + 4 services. Un simple double-clic sur `lancer-demo.command` démarre tout.

Kubernetes (Minikube) — prêt

Dossier `k8s/` avec Deployment + Service par microservice, HPA (auto-scaling 1→5 pods), sondes `/actuator/health` . À déployer avec `bash k8s/deploy.sh` .

Tests JUnit + H2

51 tests verts au total : player (1), score (28), dict (22). H2 en mémoire pour ne pas dépendre de MySQL lors des tests.

8 Check-list des consignes du sujet

Consigne	Statut	Où
Architecture microservices	✓	4 services autonomes
Spring Boot 4 / Java 26	✓	Tous les <code>pom.xml</code>
ORM / JPA / Hibernate	✓	Entités + JpaRepository partout
API REST (Richardson niveau 2)	✓	Contrôleurs @RestController
Une base par service	✓	4 schemas MySQL séparés
Communication REST inter-services	✓	RestTemplate (game→dict, game→score)
Front connecté aux 4 services	✓	CORS + fetch JS
Conteneurisation (Docker)	✓	4 Dockerfile + docker-compose
Orchestration K8s (Minikube)	✓	Dossier <code>k8s/</code> , prêt à déployer
Tests unitaires + intégration	✓	51 tests JUnit + Spring Boot Test
Actuator / supervision	✓	<code>/actuator/health</code> exposé
Gestion d'erreurs propre	✓	GlobalExceptionHandler → format JSON standard
Authentification	✓	Inscription + connexion (BCrypt) + invité
UI ludique et jolie	✓	Thème Duolingo, mascotte, animations
Partage / viralité	✓	Copier + WhatsApp + QR code